CS5785: Applied Machine Learning

Homework 1

Programming and Written Exercises

Course: CS5785: Applied Machine Learning

Homework: 1

Team Member(s): Zhijie Wang (zw892), Yuchen Feng (yf467)

1 Programming Exercises

1.1 Part I: House Prices

1.1.1 Data

The Kaggle files `train.csv`, `test.csv`, `sample_submission.csv`, and `data_description.txt` were stored with the notebook. The target is `SalePrice`, which appears only in the training data.

Table 1: Housing data dimensions

Dataset	Rows	Columns
Training data	1,460	81
Test data	1,459	80

1.1.2 Continuous and Categorical Features

Three continuous features are `GrLivArea`, `LotArea`, and `GarageArea`. Three categorical features are `Neighborhood`, `HouseStyle`, and `CentralAir`. Figure 1 shows the distribution of `GrLivArea` and the frequency of each `HouseStyle`.

Listing 1: Code used to generate the feature distributions

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].hist(train['GrLivArea'], bins=30, color='#4C78A8', edgecolor='white')
axes[0].set_title('Continuous feature: GrLivArea')
axes[0].set_xlabel('Above-ground living area (sq ft)')
axes[0].set_ylabel('Number of houses')

train['HouseStyle'].value_counts().sort_index().plot(
    kind='bar', ax=axes[1], color='#F58518', edgecolor='white'
)
axes[1].set_title('Categorical feature: HouseStyle')
axes[1].set_xlabel('House style')
axes[1].set_ylabel('Number of houses')
axes[1].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()
```

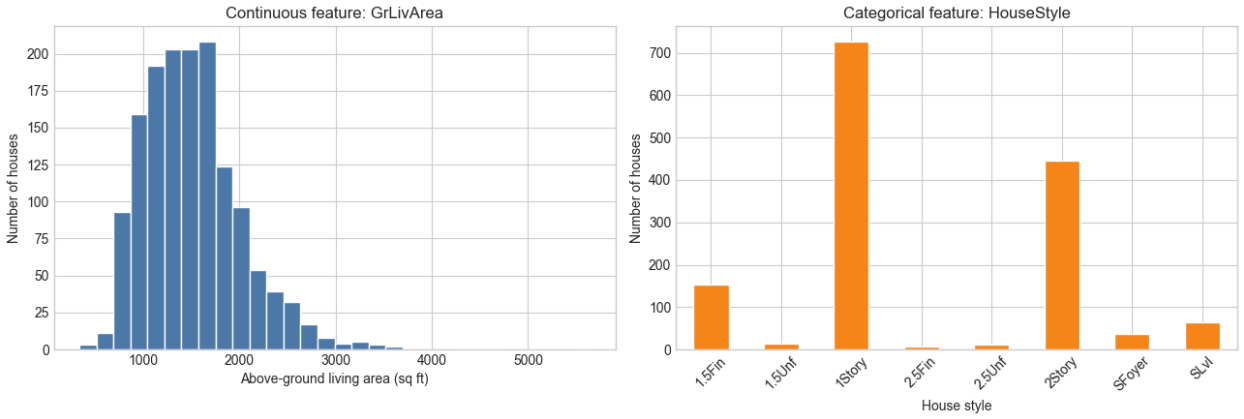


Figure 1: Distributions of a continuous feature and a categorical feature.

Most houses have a moderate living area, while a few houses have much larger values, so `GrLivArea` is right-skewed. The most common house styles are `1Story` and `2Story`.

1.1.3 Data Preprocessing

The preprocessing steps were:

1. Remove `Id` and separate `SalePrice`.
2. Treat `MSSubClass` and `MoSold` as categorical variables, even though they are stored as numbers.
3. Fill missing numerical values with training medians.
4. Fill missing categorical values with a separate `Missing` level.
5. Standardize numerical features using training means and standard deviations.
6. One-hot encode categorical features and align the test columns with the training columns.

Listing 2: Core housing preprocessing code

```
X_raw = train.drop(columns=['SalePrice', 'Id']).copy()
X_test_raw = test.drop(columns=['Id']).copy()
y = train['SalePrice'].astype(float).copy()

numeric_coded_categories = ['MSSubClass', 'MoSold']
for column in numeric_coded_categories:
    X_raw[column] = X_raw[column].astype('string')
    X_test_raw[column] = X_test_raw[column].astype('string')

numeric_columns = X_raw.select_dtypes(include='number').columns.tolist()
categorical_columns = X_raw.select_dtypes(exclude='number').columns.tolist()
assert set(numeric_coded_categories).issubset(categorical_columns)

X_numeric = X_raw[numeric_columns].copy()
X_test_numeric = X_test_raw[numeric_columns].copy()
numeric_medians = X_numeric.median()
X_numeric = X_numeric.fillna(numeric_medians)
X_test_numeric = X_test_numeric.fillna(numeric_medians)

numeric_means = X_numeric.mean()
numeric_stds = X_numeric.std(ddof=0).replace(0, 1)
X_numeric = (X_numeric - numeric_means) / numeric_stds
X_test_numeric = (X_test_numeric - numeric_means) / numeric_stds

```

```
X_categorical = X_raw[categorical_columns].fillna('Missing')
X_test_categorical = X_test_raw[categorical_columns].fillna('Missing')
X_categorical = pd.get_dummies(X_categorical, dtype=float)
X_test_categorical = pd.get_dummies(X_test_categorical, dtype=float)
X_test_categorical = X_test_categorical.reindex(columns=X_categorical.columns, fill_value=0.0)

X_processed = pd.concat([X_numeric, X_categorical], axis=1)
X_test_processed = pd.concat([X_test_numeric, X_test_categorical], axis=1)
```

The processed training and test matrices both contained 328 features. There were 34 numerical columns and 45 categorical columns before one-hot encoding, and no missing values remained after preprocessing.

1.1.4 One-Hot Encoding

Features such as `Neighborhood`, `HouseStyle`, and `Exterior1st` have no natural ordering. Assigning ordinary integers to their categories would create artificial distances. One-hot encoding creates a separate binary feature for each category.

`CentralAir` was used as a small example:

Original value	`CentralAir_N`	`CentralAir_Y`
N	1	0
Y	0	1

Listing 3: One-hot encoding and code used to generate its figure

```
central_air_ohe = pd.get_dummies(train['CentralAir'], prefix='CentralAir', dtype=int)
central_air_mapping = pd.concat([train[['CentralAir']], central_air_ohe], axis=1)
display(central_air_mapping.drop_duplicates().sort_values('CentralAir').reset_index(drop=True))

fig, axes = plt.subplots(1, 3, figsize=(14, 4))
train['CentralAir'].value_counts().sort_index().plot(
    kind='bar', ax=axes[0], color='#54A24B', edgecolor='white'
)
axes[0].set_title('Original CentralAir')
axes[0].set_xlabel('Category')
axes[0].set_ylabel('Number of houses')
axes[0].tick_params(axis='x', rotation=0)

for ax, column, color in zip(axes[1:], central_air_ohe.columns, ['#E45756', '#72B7B2']):
    ax.hist(central_air_ohe[column], bins=[-0.5, 0.5, 1.5], color=color, edgecolor='white')
    ax.set_xticks([0, 1])
    ax.set_title(column)
    ax.set_xlabel('Binary value')
    ax.set_ylabel('Number of houses')

plt.tight_layout()
plt.show()
```

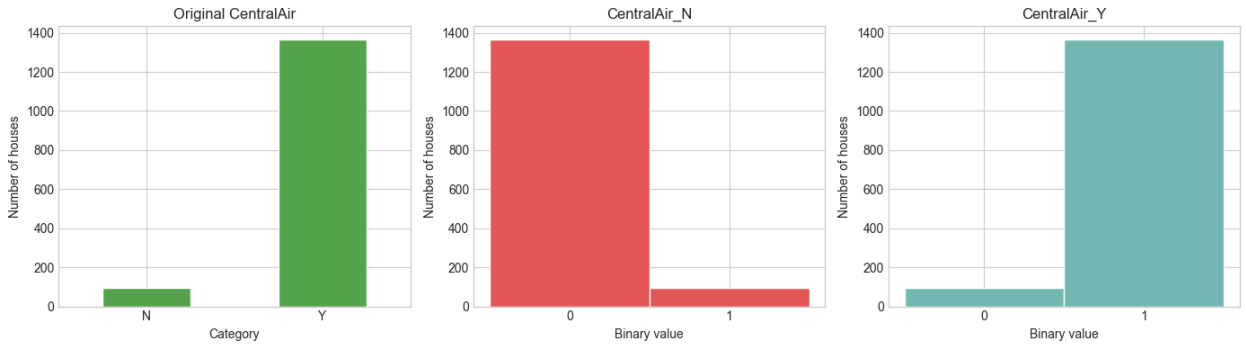


Figure 2: Original `CentralAir` distribution and its one-hot encoded columns.

A house with central air is represented by $(0, 1)$, while a house without central air is represented by $(1, 0)$. With an intercept, one of these two columns would normally be dropped because one column is determined by the other.

1.1.5 Ordinary Least Squares

I chose eight numerical features that I expected to be related to house price: `OverallQual`, `GrLivArea`, `GarageCars`, `GarageArea`, `TotalBsmtSF`, `1stFlrSF`, `FullBath`, and `YearBuilt`.

These features describe the quality, size, garage, basement, bathrooms, and age of a house. I excluded `Id` because it is only an identifier. I also limited the number of features to keep the from-scratch OLS implementation manageable.

The OLS estimate is

$$\widehat{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}.$$

The implementation calculated $\mathbf{X}^T\mathbf{X}$ and $\mathbf{X}^T\mathbf{y}$ using Python loops and solved the normal equations with Gauss-Jordan elimination and partial pivoting.

Listing 4: Gauss-Jordan solver used by OLS

```python
def gauss_jordan_solve(matrix, vector, tolerance=1e-12):
    n = len(vector)
    augmented = [list(map(float, matrix[i])) + [float(vector[i])] for i in range(n)]

    for column in range(n):
        pivot_row = max(range(column, n), key=lambda row: abs(augmented[row][column]))
        if abs(augmented[pivot_row][column]) < tolerance:
            raise ValueError('The normal-equation matrix is singular.')
        augmented[column], augmented[pivot_row] = augmented[pivot_row], augmented[column]

        pivot = augmented[column][column]
        augmented[column] = [value / pivot for value in augmented[column]]

        for row in range(n):
            if row == column:
                continue
            factor = augmented[row][column]
            if factor != 0:
                augmented[row] = [
                    augmented[row][j] - factor * augmented[column][j]
                    for j in range(n + 1)
                ]

```

```
    return [augmented[i][-1] for i in range(n)]
```

Listing 5: From-scratch OLS model

```
class OrdinaryLeastSquaresScratch:
    def fit(self, X, y_values):
        rows = [[1.0] + [float(value) for value in row] for row in X]
        targets = [float(value) for value in y_values]
        n_columns = len(rows[0])

        xtx = [[0.0 for _ in range(n_columns)] for _ in range(n_columns)]
        xty = [0.0 for _ in range(n_columns)]
        for row, target in zip(rows, targets):
            for j in range(n_columns):
                xty[j] += row[j] * target
                for k in range(n_columns):
                    xtx[j][k] += row[j] * row[k]

        self.coefficients_ = gauss_jordan_solve(xtx, xty)
        return self

    def predict(self, X):
        predictions = []
        for row in X:
            values = [1.0] + [float(value) for value in row]
            predictions.append(sum(coef * value for coef, value in zip(self.coefficients_, values))
    )
        return predictions
```

The training metrics were calculated with the following definitions:

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2, \qquad R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \widehat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \overline{y})^2}.$$

Table 2: Housing training results

Metric	Value
MSE	1,448,659,194.34
RMSE	38,061.26
R^2	0.7703

Table 3: OLS coefficients for standardized features

Term	Coefficient
Intercept	180,921.195890
OverallQual	28,934.448763
GrLivArea	26,866.711303
GarageCars	7,929.782900
GarageArea	3,112.841020
TotalBsmtSF	8,450.935814
1stFlrSF	5,389.894368
FullBath	-2,994.565034
YearBuilt	10,437.779991

Listing 6: Code used to generate the OLS diagnostic plots

```
residuals = [float(actual) - predicted for actual, predicted in zip(y, train_predictions)]

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].scatter(y, train_predictions, alpha=0.45, s=18, color='#4C78A8')
plot_min = min(float(y.min()), min(train_predictions))
plot_max = max(float(y.max()), max(train_predictions))
axes[0].plot([plot_min, plot_max], [plot_min, plot_max], '--', color='#E45756', linewidth=2)
axes[0].set_title('Training predictions vs. actual prices')
axes[0].set_xlabel('Actual SalePrice ($)')
axes[0].set_ylabel('Predicted SalePrice ($)')

axes[1].hist(residuals, bins=35, color='#72B7B2', edgecolor='white')
axes[1].axvline(0, color='#E45756', linestyle='--', linewidth=2)
axes[1].set_title('Training residual distribution')
axes[1].set_xlabel('Actual - predicted ($)')
axes[1].set_ylabel('Number of houses')

plt.tight_layout()
plt.show()
```



Figure 3: Training predictions against actual prices and the residual distribution.

These are training-set scores, as requested. The model follows the general price pattern but makes larger errors for some expensive houses. The notebook notes that this may result from outliers, omitted features, or non-linear relationships.

1.1.6 Housing Test Predictions

The model was trained with all 1,460 training rows. The same training medians, means, and standard deviations were applied to all 1,459 test rows. Negative predictions were replaced with zero, and the predictions were saved in the Kaggle submission format.

Listing 7: Housing test predictions

```
test_predictions = ols.predict(X_test_model.values.tolist())
test_predictions = [max(0.0, prediction) for prediction in test_predictions]

submission = pd.DataFrame({
    'Id': test['Id'].astype(int),
    'SalePrice': test_predictions
})
submission_path = DATA_DIR / 'submission_part1.csv'
submission.to_csv(submission_path, index=False)

assert list(submission.columns) == ['Id', 'SalePrice']
assert len(submission) == len(test) == len(sample_submission)
assert submission['Id'].equals(sample_submission['Id'])
assert not submission.isna().any().any()
```

Table 4: First five housing test predictions

Id	SalePrice
1461	114,255.467603
1462	164,857.911527
1463	167,088.495756
1464	186,843.093404
1465	222,338.312800

1.1.7 Kaggle Submission Result

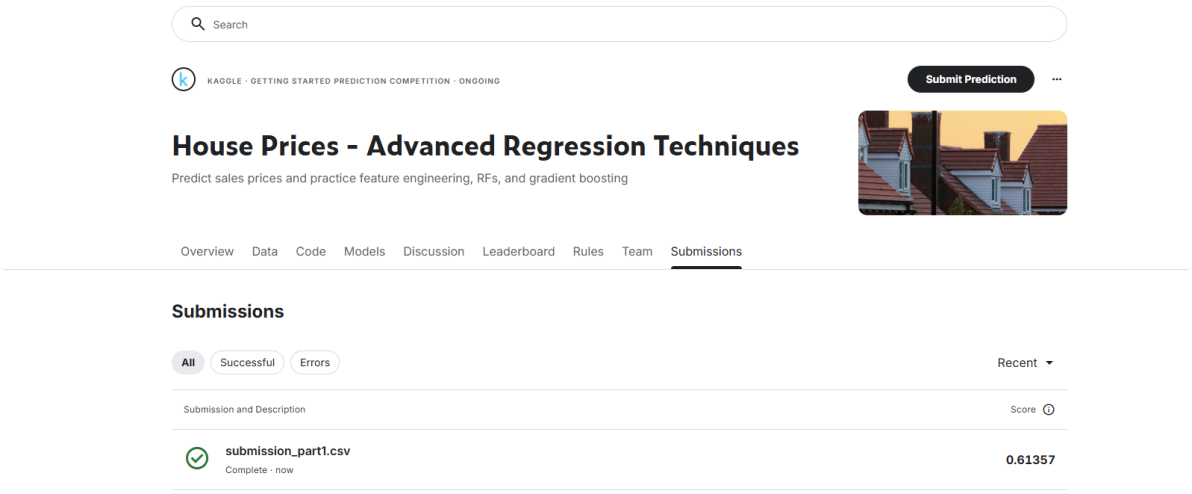


Figure 4: Successful Kaggle submission of `submission_part1.csv`.

The Kaggle submission achieved a score of 0.61357, measured by RMSE on logarithmic prices (lower is better). This score is not directly comparable to the training R^2.

The limited performance may result from using only eight numerical features and an OLS model that cannot capture non-linear relationships. Additionally, four negative predictions were clipped to zero, potentially causing large errors. Using log-transformed targets, more features, and validation could improve performance.

1.2 Part II: Titanic Disaster

1.2.1 Data and Missing Values

The training data were loaded from `titanic_train.csv`, the test data from `titanic_test.csv`, and the submission template from `gender_submission.csv`.

Table 5: Titanic missing values

Feature	Training missing	Test missing
Age	177	86
Cabin	687	327
Embarked	2	0
Fare	0	1

`Cabin` and `Age` had the most missing values. Missing data were handled in the preprocessing pipeline instead of deleting rows.

1.2.2 Feature Engineering and Preprocessing

The original features were `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`, and `Embarked`. The following feature combinations were also used:

- `FamilySize` and `IsAlone`, which describe whether a passenger traveled with family;
- `Title`, extracted from `Name` and grouped into common titles and `Rare`;
- `CabinKnown`, which records whether cabin information is present.

`PassengerId`, the original `Name`, `Ticket`, and the raw `Cabin` column were not used because they are identifiers, have many unique values, or contain too much missing data.

Listing 8: Titanic feature engineering

```
def engineer_features(frame):
    result = frame.copy()
    result['FamilySize'] = result['SibSp'] + result['Parch'] + 1
    result['IsAlone'] = (result['FamilySize'] == 1).astype(int)
    result['Title'] = result['Name'].str.extract(r',\s*([^.]*)\.', expand=False).str.strip()
    common_titles = {'Mr', 'Mrs', 'Miss', 'Master'}
    result['Title'] = result['Title'].where(result['Title'].isin(common_titles), 'Rare')
    result['CabinKnown'] = result['Cabin'].notna().astype(int)
    return result

train_features = engineer_features(train)
test_features = engineer_features(test)
```

Numerical missing values were filled with the median and then standardized. Categorical missing values were filled with the most frequent value and then one-hot encoded. The preprocessing steps and logistic regression were combined in one scikit-learn pipeline.

Listing 9: Titanic preprocessing and logistic regression

```
NUMERIC_FEATURES = [
    'Age', 'SibSp', 'Parch', 'Fare',
    'FamilySize', 'IsAlone', 'CabinKnown'
]

```

```
CATEGORICAL_FEATURES = [
    'Pclass', 'Sex', 'Embarked', 'Title'
]

numeric_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_pipeline = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(transformers=[
    ('numeric', numeric_pipeline, NUMERIC_FEATURES),
    ('categorical', categorical_pipeline, CATEGORICAL_FEATURES)
])

model = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))
])
```

1.2.3 Training and Test Predictions

The pipeline was fitted using all 891 training rows. It then predicted `Survived` for all 418 test passengers and saved the predictions in Kaggle's required format.

Listing 10: Titanic model training and submission

```
model.fit(X, y)
test_predictions = model.predict(X_test).astype(int)

submission = pd.DataFrame({
    'PassengerId': test['PassengerId'].astype(int),
    'Survived': test_predictions
})
submission_path = DATA_DIR / 'titanic_submission.csv'
submission.to_csv(submission_path, index=False)

assert list(submission.columns) == ['PassengerId', 'Survived']
assert len(submission) == len(test) == len(submission_template)
assert submission['PassengerId'].equals(submission_template['PassengerId'])
assert set(submission['Survived'].unique()).issubset({0, 1})
assert not submission.isna().any().any()
```

Table 6: First five Titanic test predictions

PassengerId	Survived
892	0
893	1
894	0
895	0
896	1

1.2.4 Kaggle Submission Result

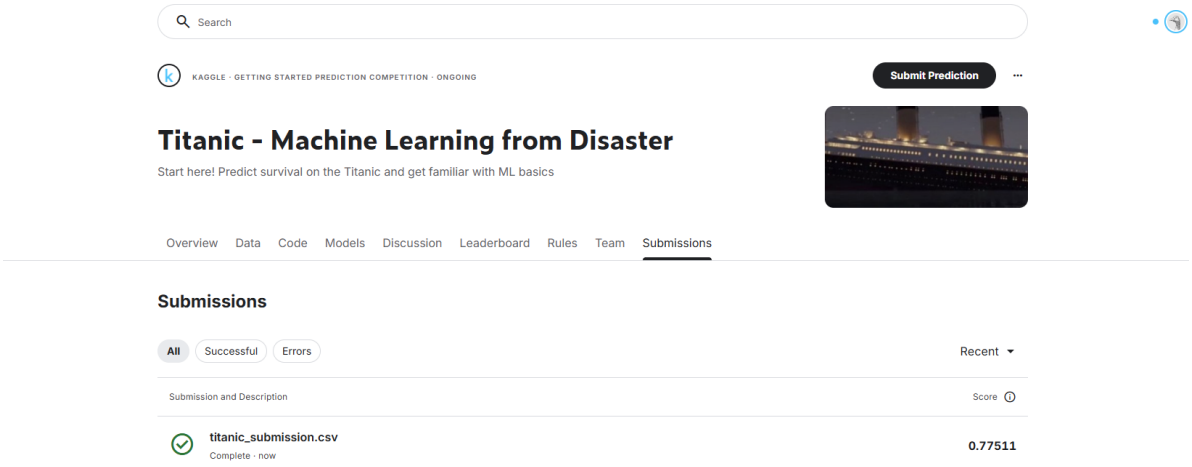


Figure 5: Successful Kaggle submission of `titanic_submission.csv`.

The Titanic submission achieved an accuracy of 0.77511 (77.5%), suggesting that the engineered features captured useful survival patterns. Remaining errors may result from the limitations of logistic regression and simple preprocessing.

2 Written Exercises

2.1 Conceptual Questions

(a) One benefit of gradient descent is that it does not need to calculate and invert the matrix $X^T X$. This is helpful when the dataset is large because working with a very large matrix can take a lot of time and memory. One drawback is that gradient descent does not give the answer in one step. We need to choose a learning rate and run several updates, and a poor learning rate can make the method very slow or prevent it from converging. The normal equations give a direct solution, but they can become expensive when the number of features is large.

(b) For a residual $r_i = y_i - \hat{y}_i$, MSE uses r_i^2, while MAE uses $|r_i|$. Since MSE squares the error, one very large error can have a much bigger effect on the final loss than the other errors. MAE only grows linearly with the size of the error. For this reason, MAE is usually more robust to outliers in the labels.

(c) The weight of the ith training point is

$$w^{(i)} = \exp\left(-\frac{(x^{(i)} - x)^2}{2\tau^2}\right).$$

As $\tau \to \infty$, the denominator $2\tau^2$ becomes very large, so the fraction inside the exponential goes to zero. Therefore,

$$w^{(i)} \longrightarrow e^0 = 1.$$

This means that all training points receive the same weight, even if some of them are far away from the query point x. Therefore, the locally weighted model becomes the regular global linear regression model fitted by ordinary least squares.

(d)(i) The polynomial features contain all monomials whose total degree is at most p. If the original data has d attributes, the number of these features, including the constant term, is

$$m = \binom{d+p}{p}.$$

For a fixed degree p, this is $O(d^p)$. If d is fixed instead, it is $O(p^d)$.

(d)(ii) Let $\Phi \in \mathbb{R}^{n \times m}$ be the design matrix after creating the polynomial features. Computing $\Phi^T \Phi$ costs $O(nm^2)$, and solving the resulting $m \times m$ system costs $O(m^3)$. Therefore, the total training cost using the normal equations is

$$O(nm^2 + m^3), \qquad m = \binom{d+p}{p}.$$

Using the estimate $m = O(d^p)$ for a fixed p, this can also be written as

$$O(nd^{2p} + d^{3p}).$$

(d)(iii) The main issue is that the number of polynomial features can grow very quickly. For example, adding more original features or increasing the degree can create many new interaction terms. This makes the model slower to train and also makes overfitting more likely. Because of this, I would mainly use polynomial regression when the number of input features and the degree are both fairly small. I would also use validation to choose the degree, and use regularization if the model starts to overfit.

2.2 Newton–Raphson for Logistic Regression

For shorter notation, let

$$p_i = p(x^{(i)}) = \sigma(\theta x^{(i)}).$$

(a) First, multiplying the numerator and denominator of the sigmoid by $e^{\theta x^{(i)}}$ gives

$$p_i = \frac{e^{\theta x^{(i)}}}{1 + e^{\theta x^{(i)}}}, \qquad 1 - p_i = \frac{1}{1 + e^{\theta x^{(i)}}}.$$

The log-likelihood is

$$\begin{aligned}
\ell(\theta) &= \sum_{i=1}^{n} \left[y^{(i)} \log p_i + (1 - y^{(i)}) \log(1 - p_i) \right] \\
&= \sum_{i=1}^{n} \left[y^{(i)} \left(\theta x^{(i)} - \log(1 + e^{\theta x^{(i)}}) \right) - (1 - y^{(i)}) \log(1 + e^{\theta x^{(i)}}) \right] \\
&= \sum_{i=1}^{n} \left[y^{(i)} \theta x^{(i)} - \log(1 + e^{\theta x^{(i)}}) \right].
\end{aligned}$$

The two log terms in the second line have the same denominator term, so they combine into one copy of $\log(1 + e^{\theta x^{(i)}})$ in the last line.

(b) We use the identity

$$\sigma'(a) = \sigma(a)(1 - \sigma(a)).$$

Since $p_i = \sigma(\theta x^{(i)})$, the chain rule gives

$$\frac{\partial p_i}{\partial \theta} = x^{(i)} p_i (1 - p_i).$$

Starting with

$$f(\theta) = \sum_{i=1}^{n} x^{(i)} (y^{(i)} - p_i),$$

we get

$$\begin{aligned}
f'(\theta) &= -\sum_{i=1}^{n} x^{(i)} \frac{\partial p_i}{\partial \theta} \\
&= -\sum_{i=1}^{n} (x^{(i)})^2 p_i (1 - p_i).
\end{aligned}$$

The negative sign appears because p_i is subtracted in $f(\theta)$.

(c) At iteration t, define

$$p_i^{(t)} = \sigma(\theta_t x^{(i)}), \qquad w_i^{(t)} = p_i^{(t)}(1 - p_i^{(t)}).$$

The Newton–Raphson update is

$$\theta_{t+1} = \theta_t - \frac{f(\theta_t)}{f'(\theta_t)}.$$

After substituting the expressions for f and f', this becomes

$$\theta_{t+1} = \theta_t + \frac{\sum_{i=1}^n x^{(i)}(y^{(i)} - p_i^{(t)})}{\sum_{i=1}^n (x^{(i)})^2 w_i^{(t)}}.$$

Let W_t be a diagonal matrix with $w_i^{(t)}$ on its diagonal. To write the same update as weighted least squares, the entries of the working response z should be

$$\boxed{z^{(i)} = \theta_t x^{(i)} + \frac{y^{(i)} - p_i^{(t)}}{p_i^{(t)}(1 - p_i^{(t)})}} \qquad (i = 1, \ldots, n).$$

To check this result, we can multiply by $x^T W_t$:

$$x^T W_t z = \theta_t x^T W_t x + \sum_{i=1}^n x^{(i)}(y^{(i)} - p_i^{(t)}).$$

Multiplying both sides by $(x^T W_t x)^{-1}$ gives the same update for θ_{t+1}. This is why Newton–Raphson for logistic regression can be viewed as an iteratively reweighted least-squares method.

2.3 Maximum Likelihood Estimation

Let $C = \{R, B, G, Y\}$. We use $y = 1$ for `goal` and $y = 0$ for `no_goal`.

(a)(i) Each ball color has its own probability of making the goal. For a color $c \in C$, define

$$\theta_c = P(Y = 1 \mid X = c), \qquad 0 \le \theta_c \le 1.$$

We can model the result of one throw with a Bernoulli distribution:

$$p(y \mid x = c; \theta) = \theta_c^y (1 - \theta_c)^{1-y},$$

where

$$\theta = (\theta_R, \theta_B, \theta_G, \theta_Y).$$

This model assumes that the throws are independent once the ball color is known.

(a)(ii) This is a supervised learning problem. The ball color is the input, and the observed result, either `goal` or `no_goal`, is the label. Since both the input and the label are included in the data, the model can learn the relationship between them.

(b) Let s_c be the number of goals for color c, and let f_c be the number of misses. The log-likelihood is

$$\ell(\theta) = \sum_{c \in C} \left[s_c \log \theta_c + f_c \log(1 - \theta_c) \right].$$

The counts from the data are

c	R	B	G	Y
s_c	1	3	1	0
f_c	2	0	0	1

so the log-likelihood for this dataset is

$$\ell(\theta) = \log \theta_R + 2 \log(1 - \theta_R) + 3 \log \theta_B + \log \theta_G + log(1 - \theta_Y).$$

The likelihood tells us how likely the recorded results are for a particular choice of the four probabilities. Therefore, maximizing it chooses the probabilities that best agree with the data we observed. Using the logarithm also changes the product of many probabilities into a sum, which is easier to work with.

(c) For each color, let $n_c = s_c + f_c$. When there is at least one goal and one miss, we can set the derivative of that color's log-likelihood equal to zero:

$$\frac{s_c}{\theta_c} - \frac{f_c}{1 - \theta_c} = 0.$$

Solving this equation gives

$$\widehat{\theta}_c = \frac{s_c}{n_c}.$$

This result is just the observed fraction of successful throws. It also works for the boundary cases. If every recorded throw is a goal, the estimate is 1. If every recorded throw is a miss, the estimate is 0. Therefore,

$$\boxed{\hat{\theta}_R = \frac{1}{3}, \quad \hat{\theta}_B = 1, \quad \hat{\theta}_G = 1, \quad \hat{\theta}_Y = 0.}$$

For the red ball, one out of three throws was a goal, so its estimated probability is 1/3. All three blue throws were goals, so the estimate for blue is 1. The green ball had one goal and the yellow ball had one miss, giving estimates of 1 and 0. However, the green and yellow estimates each come from only one observation, so they are not very reliable.

(d) These estimates may be inaccurate when there are only a few observations. For example, the green ball was used only once and happened to make the goal. Its MLE is therefore 1, but this does not prove that the green ball will always make the goal. Its true probability could be 0.6, and one successful throw would still be possible. The data could also be misleading if the player gets better over time or if the game conditions change, because the model assumes that each color always has the same success probability.

3 Summary

For House Prices, we implemented OLS from scratch using eight numerical features, achieving a training R^2 of 0.7703 and a Kaggle score of 0.61357. The results suggest that additional features and improved modeling could enhance performance.

For Titanic, we applied feature engineering and logistic regression with a preprocessing pipeline, achieving a Kaggle accuracy of 0.77511. Both models produced valid submissions, but their performance could be improved through further feature engineering and model refinement.

4 AI Usage Statement

We only used generative AI to learn machine learning concepts such as preprocessing, one-hot encoding, ordinary least squares, logistic regression, feature engineering, and scikit-learn pipelines.